

REDVAULT PASSWORTMANAGER

SICHER. EINFACH. LOCAL.



17.06.2026
Yuliya Krause



Inhalte

1. Ziel des Projekts RedVault
2. Technologien
3. Architektur RedVault
4. Umgesetzte Features
5. Erweiterungsmöglichkeiten Passwortmanager
6. Passwortstärke Anzeige
7. Logik & Verschlüsselung
8. Herausforderungen
9. Live-Demo

Ziel des Projekts RedVault

Entwicklung eines sicheren, lokalen Passwortmanagers in Python

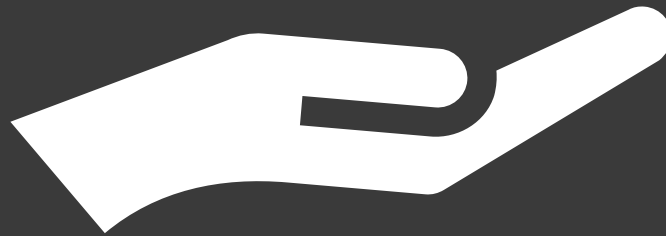


Sichere
Verwaltung
Website-
Logins

Lokale &
verschlüsselte
Speicherung

Einfacher
Zugang

Moderne UI



Technologien

Cryptography

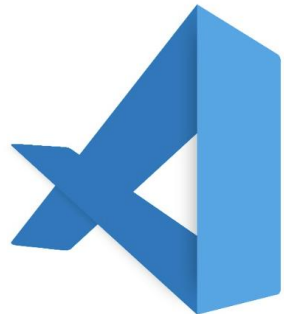


REDVAULT

SQLModel

SQLite

Visual Studio



GitHub



CustomTkinter (UI)



Python

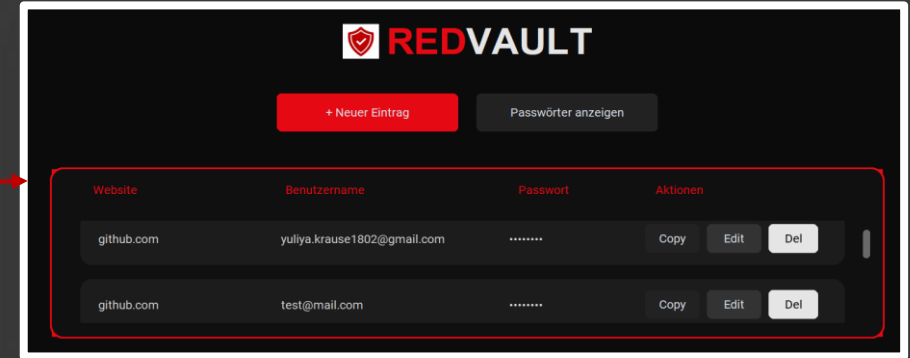


Architektur - RedVault



FRONTEND (UI)

GUI mit CustomTkinter
Benutzerinteraktion (Buttons, Eingaben)
Anzeige der gespeicherten Passwörter



BACKEND / LOGIK

Verarbeitung der Daten
Verschlüsselung & Entschlüsselung
Steuerung der Anwendung

```
class VaultService:
    def __init__(self, repository, crypto_service):
        self.repository = repository
        self.crypto_service = crypto_service

    def add_password(self, website, username, password, category="", notes=""):
        encrypted_password = self.crypto_service.encrypt(password)

        entry = PasswordEntry(
            website,
            username,
            encrypted_password,
            category,
            notes
        )
```



DATENBANK (SQLITE)

Lokale Speicherung
Tabelle: password_entries
CRUD-Operationen

```
import sqlite3

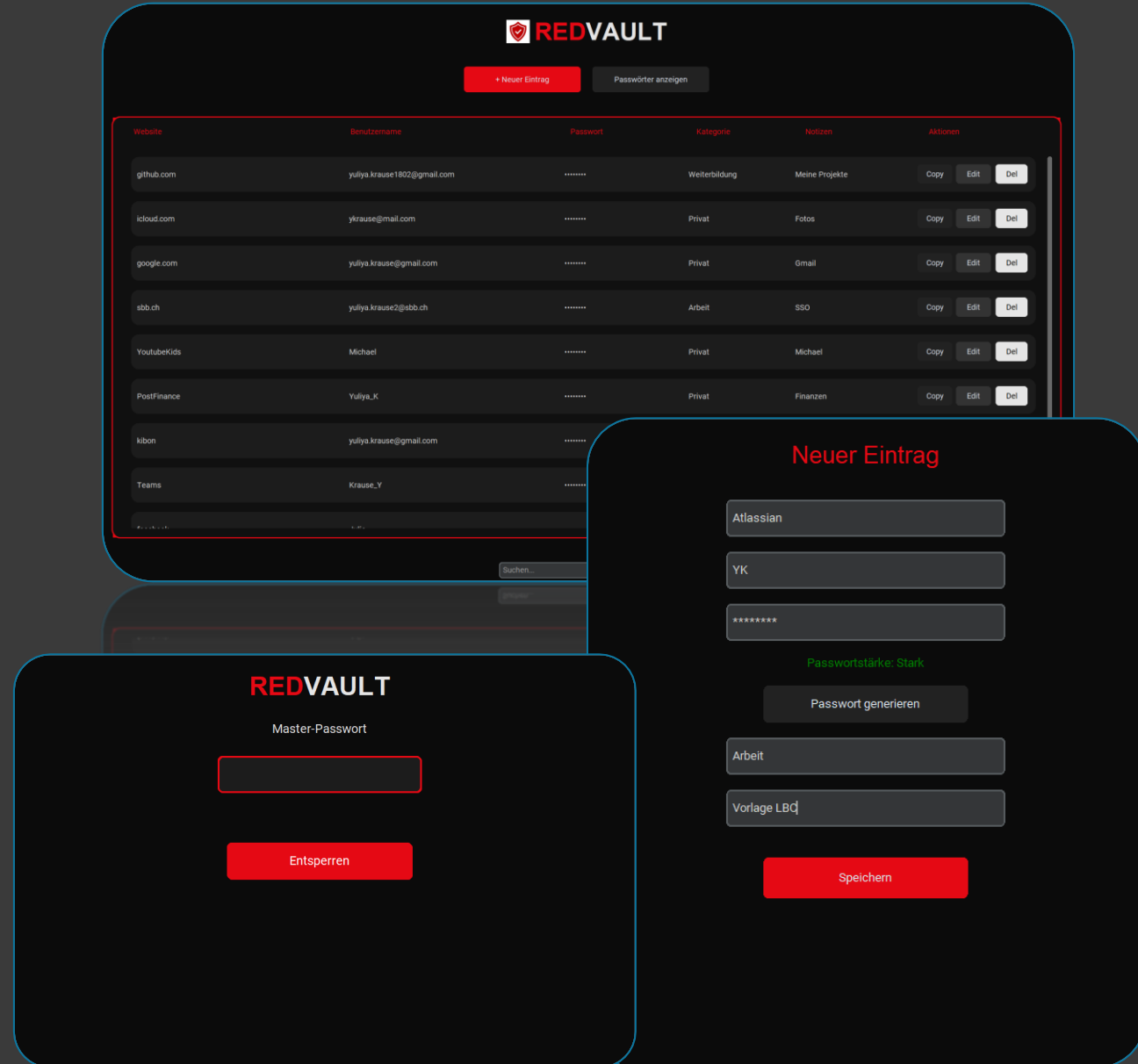
class Database:
    def __init__(self, db_name="vault.db"):
        self.connection = sqlite3.connect(db_name)
        self.create_tables()

    def create_tables(self):
        cursor = self.connection.cursor()
```



Umgesetzte Features

- 1 Anmeldung mit Master-Passwort
- 2 CRUD-Funktionen für Passwort-Einträge
- 3 AES-basierter Fernet-Verschlüsselung
- 4 Passwortgenerator & Passwortstärke Anzeige
- 5 Passwortmaskierung
- 6 Copy-to-Clipboard Funktion
- 7 Suche nach Website oder Benutzername
- 8 Modulare OOP-Aufbau



Erweiterungsmöglichkeiten Passwortmanager

Verschlüsselung

- Ende-zu-Ende-Verschlüsselung: Nur der Nutzer kann die Daten lesen
- Auto-Lock bei Inaktivität



Hohe Datensicherheit

Login-System

- Anmeldung mit Benutzername + Master-Passwort
- Erstellung eigenes Master-Passwort
- 2-Faktor-Authentifizierung per App oder SMS



Synchronisation und Zugriff

Cloud Sync

- Daten werden verschlüsselt in der Cloud gespeichert
- Synchronisation zwischen PC, Smartphone und Tablet



Benutzerfreundlichkeit

- Strukturierte Organisation der Passwörter
- Favoriten-System
- Papierkorb, Icons
- Kategorie-Auswahl als Dropdown



Strukturierte Verwaltung

Passwortstärke Anzeige

```
app > services > password_generator.py > PasswordGenerator > generate
5   class PasswordGenerator:
34
35       @staticmethod
36       def check_password_strength(password):
37           score = 0
38
39           if len(password) >= 8:
40               score += 1
41
42           if any(char.isdigit() for char in password):
43               score += 1
44
45           if any(char.isupper() for char in password):
46               score += 1
47
48           if any(char.islower() for char in password):
49               score += 1
50
51           if any(char in "!@#$%^&*()" for char in password):
52               score += 1
53
54           if score <= 2:
55               return "Schwach"
56           elif score <= 4:
57               return "Mittel"
58           else:
59               return "Stark"
```

- Die Funktion prüft fünf Sicherheitskriterien:

1. Mindestens 8 Zeichen
2. Mindestens eine Zahl
3. Mindestens ein Grossbuchstabe
4. Mindestens ein Kleinbuchstabe
5. Mindestens ein Sonderzeichen

- Für jedes erfüllte Kriterium wird 1 Punkt zum Score hinzugefügt (max. 5 Punkte)
- Je nach erreichter Punktzahl wird die Passwortstärke zugegeben

- | | |
|-----------|---------------------------------|
| ● Schwach | 0 -2 Punkt → sehr unsicher |
| ● Mittel | 3 – 4 Punkte → teilweise sicher |
| ● Stark | 5 Punkte → sehr sicher |

Logik & Verschlüsselung

→ Passwort wird vor dem Speichern verschlüsselt.

```
app > services > vault_service.py > VaultService > get_passwords
1  from app.models.password_entry import PasswordEntry
2
3  class VaultService:
4      def __init__(self, repository, crypto_service):
5          self.repository = repository
6          self.crypto_service = crypto_service
7
8      def add_password(self, website, username, password, category="", notes=""):
9          encrypted_password = self.crypto_service.encrypt(password)
10
11          entry = PasswordEntry(
12              website,
13              username,
14              encrypted_password,
15              category,
16              notes
17          )
18
19          self.repository.add_entry(entry)
20
```



→ Passwort wird beim Anzeigen entschlüsselt.

→ Übergabe an die Datenbank.



```
3  class VaultService:
20
21      def get_passwords(self):
22          entries = self.repository.get_all_entries()
23          result = []
24
25          for entry in entries:
26              entry_id, website, username, encrypted_password, category, notes = entry
27              decrypted_password = self.crypto_service.decrypt(encrypted_password)
28
29              result.append((entry_id, website, username, decrypted_password, category, notes))
30
31          return result
```

Herausforderungen



main.py in mehrere Dateien und Module aufteilen

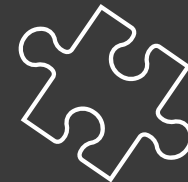
Objektorientierte Struktur planen

Sichere Passwortspeicherung umsetzen

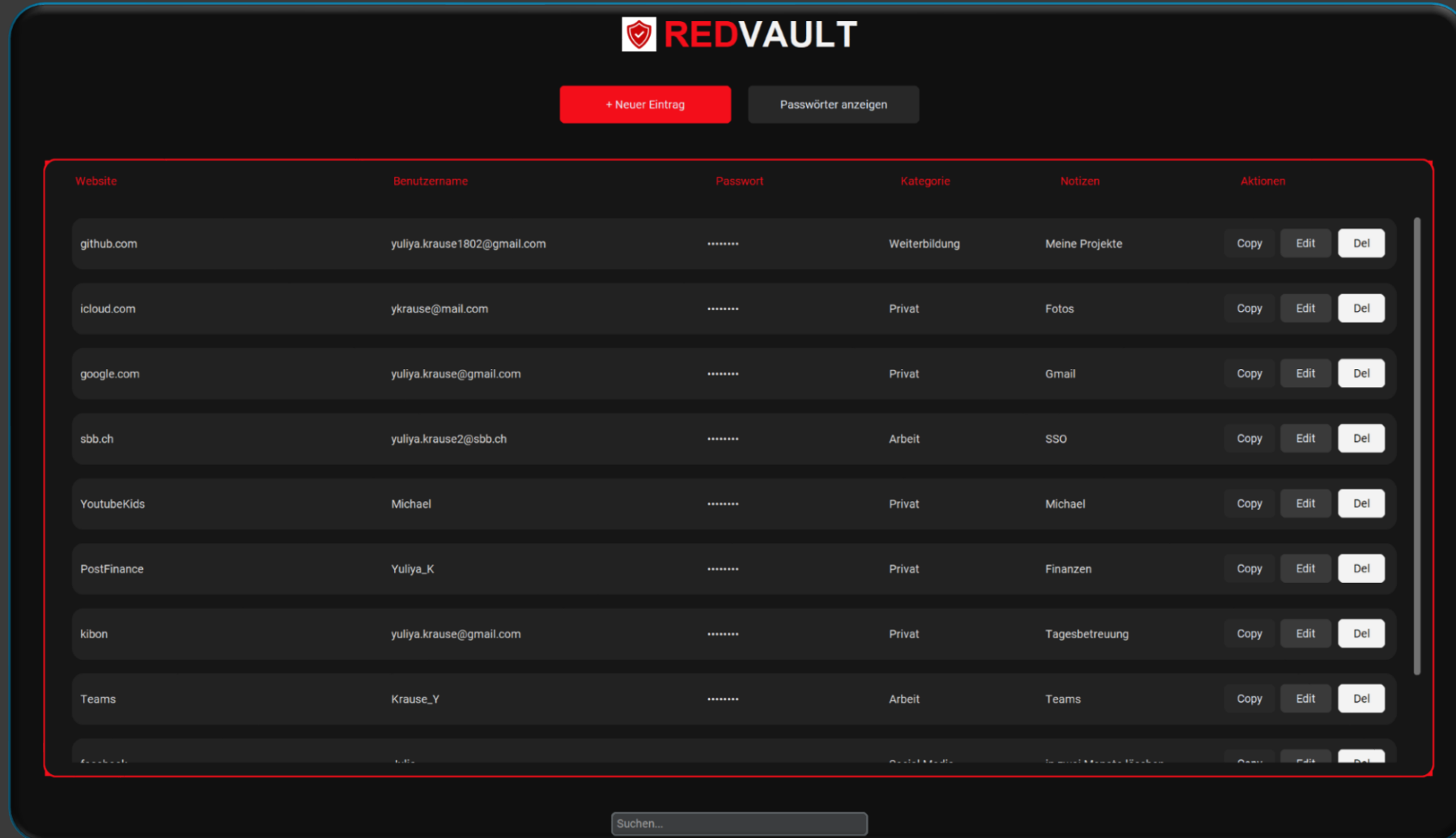
Features erweitern und bestehenden Code anpassen

Fehlerbehandlung & Eingabevalidierung

Frontend-Spalten gleichmässig anpassen



Passwortmanager in Aktion



Live-Demo

"Life is short
(You need Python)»
Bruce Eckel

Danke, merci & grazie